Outlook

[WATCH 2026] Submission #36 "TraceLock: Verified Streaming Evidence..."

From WATCH 2026 HotCRP <noreply-watch2026@hotcrp.com>
Date Fri 9/4/2026 3:02 PM
To Ghimire, Sudrit <aui25@txstate.edu>

Dear Sudrit Ghimire,

Congratulations! We are pleased to inform you that your paper has been accepted for presentation at the workshop.

Please refer to the enclosed reviews and consider the reviewers' comments while preparing the camera-ready copy. The camera-ready submission deadline is September 11, 2026. Detailed camera-ready submission instructions will be provided by ACM at a later date.

Congratulations again, and we look forward to seeing your work at the workshop!
* Title: TraceLock: Verified Streaming Evidence for Linux Telemetry
* Site: https://nam04.safelinks.protection.outlook.com/?url=https%3A%2F%2Fwatch2026.hotcrp.com%2Fpaper%2F36&data=05%7C02%7Caui25%40txstate.edu%7Cf8cdd8a7575b444886bd08df0abf7e18%7Cb19c134a14c94d4caf65c420f94c8cbb%7C0%7C0%7C639241489724981695%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=RVblSfX3nLy89zhquYOYKG4hHf%2BSGm5ibReHR%2BwwAxY%3D&reserved=0

Contact WATCH.miniconf@gmail.com with any questions or concerns.

- WATCH 2026 Submissions
https://nam04.safelinks.protection.outlook.com/?url=https%3A%2F%2Fwatch2026.hotcrp.com%2F&data=05%7C02%7Caui25%40txstate.edu%7Cf8cdd8a7575b444886bd08df0abf7e18%7Cb19c134a14c94d4caf65c420f94c8cbb%7C0%7C0%7C639241489725005731%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=lqpFvljwjEnBbZbE1JcV7rR6mdgWJUkykN%2FKvpOhrfw%3D&reserved=0

Review #36A
==========

Overall merit
---
4. Accept

Reviewer expertise
---
3. Knowledgeable

Paper summary

The paper proposes TraceLock, built around K-Sentry, for Linux telemetry, evaluated its implementation in Rust, and presented and compared results.

Comments for authors

The paper is timely, and discusses a standing issue that has been there for a while.
The presentation is good, and the readability is good.
Stable observed rates, workload overhead and resource usage are all impressive.
Limitations of the implementation and evaluation are discussed.
Are there scalability concerns or reliability issues?

* *

Review #36B
==========

Overall merit
---
3. Weak accept

Reviewer expertise
---
3. Knowledgeable

Paper summary

The manuscript introduces **TraceLock**, a verified, order-sensitive streaming evidence layer designed to guarantee the integrity of Linux telemetry event sequences post-collection. Telemetry sequences are highly susceptible to malicious post-collection tampering, including event omission, insertion, reordering, or same-length replacement—attacks that standard length checks or unordered set summaries fail to detect.

The core of TraceLock is **K-Sentry**, a finite-field accumulator that maps canonicalized system events into field elements and computes order-sensitive checkpoints using non-affine, triangular positional weights ($T_i = \frac{i(i+1)}{2}$). The non-affine exponent design prevents the algebraic shift-rebasing weakness inherent in standard affine rolling hashes, making it impossible for an attacker to rebase shifted segment digests using a single scalar independent of the segment's contents. The mathematical core is formally verified in **Verus** to ensure that the Rust-implemented accumulator conforms strictly to its specification. The authors implement a live eBPF-based collector in Rust tracing `execve`, `openat`,

and `connect` events, and evaluate the prototype on native AWS EC2 workloads, demonstrating low overhead (0.83% to 6.67%) compared to heavier runtime detectors like Falco and auditd.

Comments for authors
---
1. Misleading "Apples-to-Oranges" Baseline Comparison in Table 10
In Table 10 ("Accumulator-cost ablation at 100k events"), the authors compare the throughput of K-Sentry (665,247 events/s) with a Python-based Merkle tree baseline. The table claims the Merkle baseline achieves a massive throughput of **10,233,681 events/s** based on a runtime of **9.77 ms** for 100,000 events.
* As noted in the text, the Merkle baseline is "chunk-level" with an interval of 1,000. This means the Merkle code is only performing **100 cryptographic hashes** (one per chunk) rather than processing 100,000 events individually.
* Presenting this as "10.2 million events/s" is highly misleading. A chunk-level tree does not provide per-event fine-grained updates or order-sensitivity within the chunk without additional local storage. Comparing a chunked hash to a per-event mathematical accumulator, and then plotting them on the same "Events/s" metric, is an unscientific comparison. The authors must either run a true per-event Merkle baseline or explicitly adjust the throughput metric to reflect actual hashed blocks.

2. Physically Impossible Negative Overhead in Table 6
In the performance evaluation (Table 6), the authors list the system overhead for the `process_exec` workload under TraceLock as **`-0.31%`**.
* A software monitoring layer—which attaches eBPF kprobes, transfers data through a ring buffer, canonicalizes text strings, maps hashes to a finite field, and executes mathematical updates—cannot have a negative performance cost.
* While negative overheads can occasionally occur in systems benchmarking due to CPU cache alignments, scheduler noise, or thread-scheduling jitter, reporting a negative overhead without any explanation suggests a lack of calibration or statistical validation. The authors should explain this measurement noise or report the confidence intervals (standard deviation) over multiple runs to show that the negative value is within the margin of error.

* *

Review #36C
==========

Overall merit
---
4. Accept

Reviewer expertise
---
1. No familiarity

Paper summary

The paper introduces TraceLock, a verified order-sensitive streaming evidence layer for Linux telemetry designed to prove that an ordered telemetry sequence has been preserved after collection. TraceLock is based on a finite-field accumulator, K-Sentry, using non-affine exponents in order to prevent digest-only scalar rebasing independent of segment contents.

Comments for authors

The paper is interesting, and clearly outlines the theoretical framework, although in some places it seems a bit too concise to me, especially regarding the implementation details; it would probably be helpful to include a reference to the code used.

CAUTION:. This email originated from outside the TXST network. Do not click links or download files unless you know the sender and content are safe.